



Departamento de Informática e Matemática Aplicada – DIMAp

Comunicação HTTP

Prof. Nélio Cacho

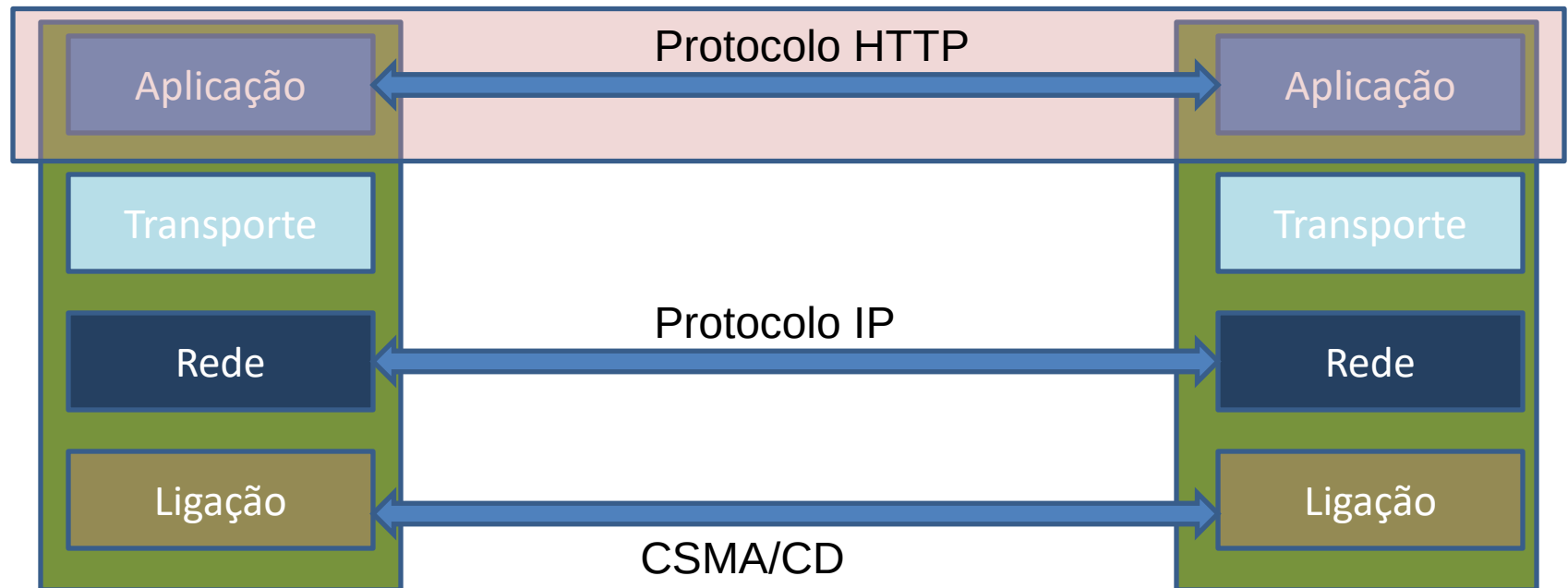
DIM0614 – Programação Distribuída

Hora de silenciar o celular...



- Manter o telefone celular sempre desligado/silencioso quando estiver em sala de aula;
- Nunca atender o celular na sala de aula.

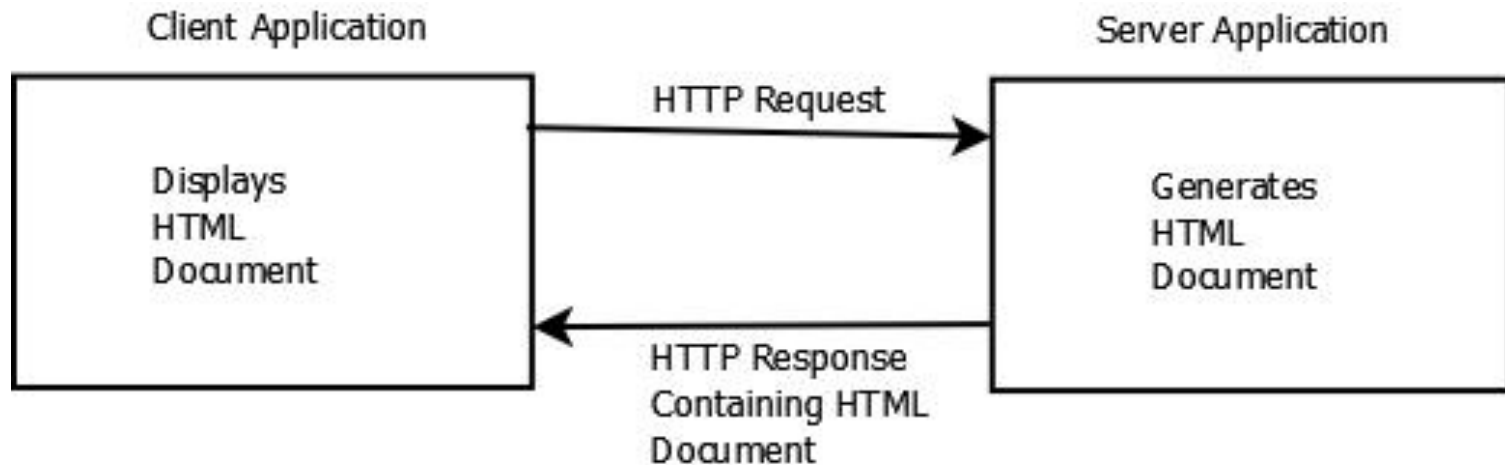
Como realizar a comunicação ?



Protocolo HTTP

- O HTTP é um protocolo de rede utilizado para entregar recursos na World Wide Web (WWW).
- A maioria das pessoas usa um navegador para se comunicar na WWW.
- O navegador é uma aplicação cliente, enquanto o servidor web responde às solicitações do cliente.
- A porta padrão utilizada por esses servidores é a 80.

Protocollo HTTP



Protocolo HTTP

- O HTTP evoluiu ao longo dos anos. O HTTP/1.0 surgiu nos anos 1980 e 1990, com a primeira documentação lançada em 1991.

A última definição do HTTP/1.1 foi lançada como uma especificação em seis partes, em junho de 2014. Um **Request For Comments** (RFC) para o HTTP 2.0 foi lançado em maio de 2015.

Version	Reference
HTTP 1.0	http://www.w3.org/Protocols/HTTP/1.0/spec.html
HTTP/1.1	http://tools.ietf.org/html/rfc2616
HTTP/2	https://en.wikipedia.org/wiki/HTTP/2

O Novo Protocolo HTTP

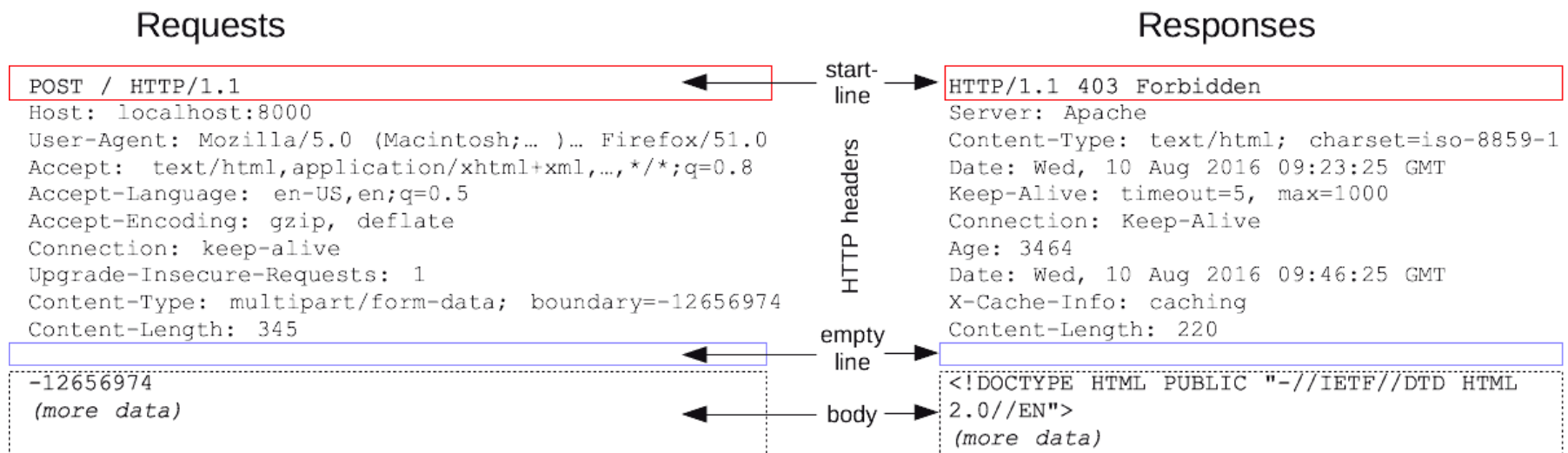
Comparison of HTTP protocol stack changes from HTTP/1.1 to HTTP/3		
HTTP 1.1	HTTP 2	HTTP 3
Some methods and response codes are added. “Keep Alive” becomes officially supported. “Host” header is supported for Virtual Domain. Syntax and semantics are separated.	Support of parallel request transmission by “stream” (addressing HoL blocking issue of HTTP requests). Addition of flow-control function in units of “Stream”. Addition of prioritization function in units of “Stream”. Addition of PUSH function (send related file without request).	Lower protocol changes from TCP+TLS to UDP+QUIC. “Streams” and “flow-control function” are prioritized. Parallel request transmission is supported by QUIC stream (addressing HoL blocking issue of TCP packets).

<https://ably.com/topic/http3>

<https://www.toptal.com/web/performance-working-with-http-3>

Mensagem HTTP

- Solicitações e respostas de mensagens HTTP utilizam esta estrutura:
 1. Uma linha indicando o tipo de mensagem;
 2. Zero ou mais linhas de cabeçalho;
 3. Uma linha em branco;
 4. Um corpo de mensagem opcional contendo dados.



Formato da Linha de Solicitação Inicial

- O formato das linhas iniciais de solicitação e resposta diferem. A linha de solicitação consiste em três partes separadas por espaços:
 - Nome do método de solicitação;
 - Caminho local do recurso desejado: descreve o recurso a ser acessado. Segue o nome do hostname na URL de requisição;
 - Versão HTTP

Exe.: **GET /index HTTP/1.0**

Request Method Name

- Os nomes dos métodos referem-se à ação solicitada pelo cliente. Eles são sempre escritos em letras maiúsculas.
Métodos do
HTTP/1.1:(<http://www.w3.org/Protocols/rfc2616/rfc2616-sec9.html>).
 - GET: Recupera informações identificadas pelo URI da solicitação.
 - POST: Insere e atualiza dados.
 - PUT: Armazena a entidade sob o URI fornecido.
 - DELETE: Solicita que o servidor apague o recurso identificado.

Formato da Linha de Resposta Inicial

- A linha de resposta inicial tem três partes separadas por espaços:

1. Versão HTTP;

2. Código de status de resposta (três dígitos):

1. 1xx: Informativo.

2. 2xx: Sucesso.

3. 3xx: Redirecionamento.

4. 4xx: Erro do cliente.

5. 5xx: Erro do servidor.

3. Frase descritiva do código.

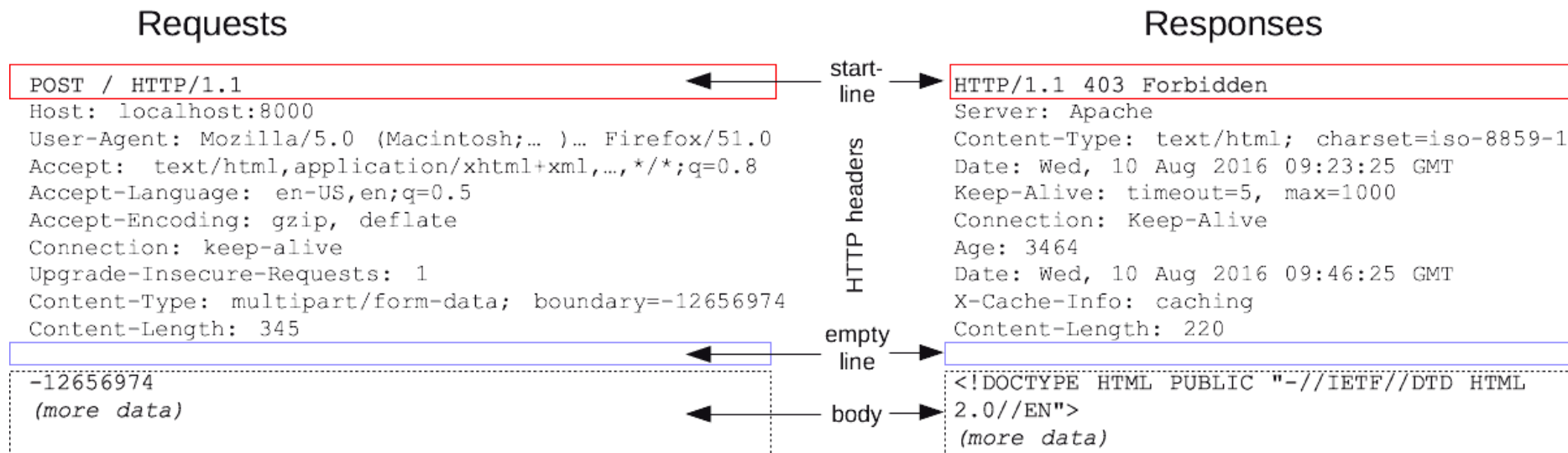
Exe.: **HTTP/1.0 404 Not Found**

Código de status de resposta

Status code	Standard text	Meaning
200	OK	This indicates that the request was a success
301	Moved Permanently	This indicates that the URL has been moved permanently and the link should be updated
302	Found	This indicates that the resource is temporarily located somewhere else, but the URL should still be used
307	Temporary Redirect	This is similar to 302 , but the method used should not be changed, which may happen with 302
308	Permanent Redirect	This is similar to 301 , but the method used should not be changed, which may happen with 301
400	Bad Request	This indicates that request access was incorrect
401	Unauthorized	This indicates that the resource is restricted often because the login attempt failed
403	Forbidden	This indicates that access to the requested resource is forbidden
404	Not Found	This indicates that the resource is no longer available
500	Internal server error	This reflects some sort of error with the server
502	Bad Gateway	This indicates that the gateway server received an invalid response from another server
503	Service Unavailable	This indicates that the server is not available
504	Gateway Timeout	This indicates that the gateway server did not receive a response from another server in a timely manner

Linhas de Cabeçalho

- As linhas de cabeçalho fornecem informações sobre a solicitação ou resposta, como o e-mail do remetente ou um identificador de aplicação.
- Formato: Identificador do cabeçalho: Valor.
- A linha de cabeçalho ocupa 1 linha



Header Lines

POST / HTTP/1.1

Host: localhost:8000

User-Agent: Mozilla/5.0 (Macintosh;...)... Firefox/51.0

Accept: text/html,application/xhtml+xml,...,*/*;q=0.8

Accept-Language: en-US,en;q=0.5

Accept-Encoding: gzip, deflate

Connection: keep-alive

Upgrade-Insecure-Requests: 1

Content-Type: multipart/form-data; boundary=-12656974

Content-Length: 345

-12656974

(more data)

Request headers

General headers

Representation
headers

Header Lines

Uma lista de campos de cabeçalho e descrições pode ser encontrada em

https://en.wikipedia.org/wiki/List_of_HTTP_header_fields.

Uma lista de strings de agentes pode ser encontrada em

<http://useragentstring.com/pages/useragentstring.php>.

O HTTP 1.0 define 16 cabeçalhos

(<http://www.w3.org/Protocols/HTTP/1.0/spec.html#HeaderFields>), enquanto o HTTP 1.1 possui 47 cabeçalhos

(<http://tools.ietf.org/html/rfc2616#section-14>).

Corpo da Mensagem

- Este é o dado que constitui a mensagem. Ele pode ou não estar presente. Quando incluído, cabeçalhos como Content-Type e Content-Length são usados para fornecer mais informações.

Content-type: text/html

Content-length: 105

- O corpo da mensagem pode aparecer assim:

```
<html><h1>HTTPServer Home Page.... </h1><br><b>Welcome to the new and improved  
web server!</b><BR></html>
```


Java Socket Support for HTTP (Server Side)

```
public class WebServer {

    public WebServer() {
        System.out.println("Webserver Started");
        try (ServerSocket serverSocket = new ServerSocket(8080)) {
            while (true) {
                System.out.println("Waiting for client request");
                Socket remote = serverSocket.accept();
                System.out.println("Connection made");
                new Thread(new ClientHandler(remote)).start();
            }
        } catch (IOException ex) {
            ex.printStackTrace();
        }
    }

    public static void main(String args[]) {
        new WebServer();
    }
}
```

Java Socket Support for HTTP (Server Side)

```
public class ClientHandler implements Runnable {  
    private final Socket socket;  
    public ClientHandler(Socket socket) {  
        this.socket = socket;  
    }  
    @Override  
    public void run() {  
        System.out.println("\nClientHandler Started for " +  
            this.socket);  
        handleRequest(this.socket);  
        System.out.println("ClientHandler Terminated for "  
            + this.socket + "\n");  
    }  
}
```

O método `handleRequest` utiliza fluxos de entrada e saída para se comunicar com o servidor. Além disso, ele determina qual solicitação foi feita e, em seguida, processa essa solicitação.

Java Socket Support for HTTP (Server Side)

```
public void handleRequest(Socket socket) {
    try (BufferedReader in = new BufferedReader(
        new InputStreamReader(socket.getInputStream()));) {
        String headerLine = in.readLine();
        StringTokenizer tokenizer = new StringTokenizer(headerLine);
        String httpMethod = tokenizer.nextToken();

        if (httpMethod.equals("GET")) {
            System.out.println("Get method processed");
            String httpQueryString = tokenizer.nextToken();
            StringBuilder responseBuffer = new StringBuilder();
            responseBuffer
                .append("<html><h1>WebServer Home Page.... </h1><br>")
                .append("<b>Bem vindo ao Meu web server!</b><BR>")
                .append("</html>");
            sendResponse(socket, 200, responseBuffer.toString());
        } else {
            System.out.println("The HTTP method is not recognized");
            sendResponse(socket, 405, "Method Not Allowed");
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

Java Socket Support for HTTP (Server Side)

```
public void sendResponse(Socket socket, int statusCode, String responseString) {
    String statusLine;
    String serverHeader = "Server: WebServer\r\n";
    String contentTypeHeader = "Content-Type: text/html\r\n";

    try (DataOutputStream out = new DataOutputStream(socket.getOutputStream())) {
        if (statusCode == 200) {
            statusLine = "HTTP/1.0 200 OK" + "\r\n";
            String contentLengthHeader = "Content-Length: " + responseString.length() + "\r\n";

            out.writeBytes(statusLine);
            out.writeBytes(serverHeader);
            out.writeBytes(contentTypeHeader);
            out.writeBytes(contentLengthHeader);
            out.writeBytes("\r\n");
            out.writeBytes(responseString);
        } else if (statusCode == 405) {
            statusLine = "HTTP/1.0 405 Method Not Allowed" + "\r\n";
            out.writeBytes(statusLine);
            out.writeBytes("\r\n");
        } else {
            statusLine = "HTTP/1.0 404 Not Found" + "\r\n";
            out.writeBytes(statusLine);
            out.writeBytes("\r\n");
        }
        out.close();
    } catch (IOException ex) {
        ex.printStackTrace();
    }
}
```

Java Socket Support for HTTP (Client Side)

```
public class HTTPClient {

    public HTTPClient() {
        System.out.println("HTTP Client Started");
        try {
            InetAddress serverInetAddress =
                InetAddress.getByName("127.0.0.1");
            Socket connection = new Socket(serverInetAddress, 80);

            try (OutputStream out = connection.getOutputStream();
                BufferedReader in =
                    new BufferedReader(new
                        InputStreamReader(
                            connection.getInputStream()))) {
                sendGet(out);
                System.out.println(getResponse(in));
            }
        } catch (IOException ex) {
            ex.printStackTrace();
        }
    }

    ...

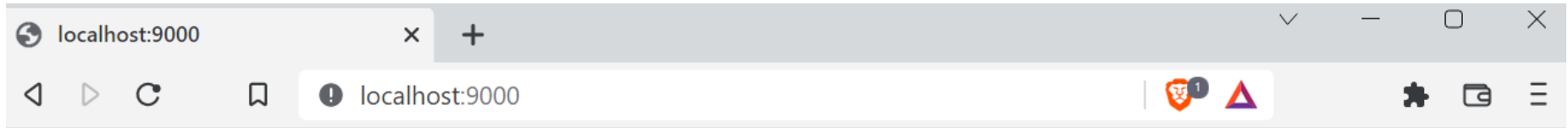
    public static void main(String[] args) {
        new HTTPClient();
    }
}
```

Java Socket Support for HTTP (Client Side)

```
private void sendGet(OutputStream out) {  
    try {  
        out.write("GET /default\r\n".getBytes());  
        out.write("User-Agent: Mozilla/5.0\r\n".getBytes());  
    } catch (IOException ex) {  
        ex.printStackTrace();  
    }  
}
```

```
private String getResponse(BufferedReader in) {  
    try {  
        String inputLine;  
        StringBuilder response = new StringBuilder();  
        while ((inputLine = in.readLine()) != null) {  
            response.append(inputLine).append("\n");  
        }  
        return response.toString();  
    } catch (IOException ex) {  
        ex.printStackTrace();  
    }  
    return "";  
}
```

Testando seu servidor



WebServer Home Page....

Welcome to my web server!

HTTP Sampler

HTTP Request

Name: HTTP Request

Comments:



Basic Advanced

Web Server

Protocol [http]: Server Name or IP: localhost Port Number: 8080

HTTP Request

GET Path: Content encoding:

☐ Redirect Automatically ☐ Follow Redirects ☐ Use KeepAlive ☐ Use multipart/form-data ☐ Browser-compatible headers



Parameters Body Data Files Upload

Send Parameters With the Request:

Name:	Value	URL Encode?	Content-Type	Include Equals?
-------	-------	-------------	--------------	-----------------

Detail

Add

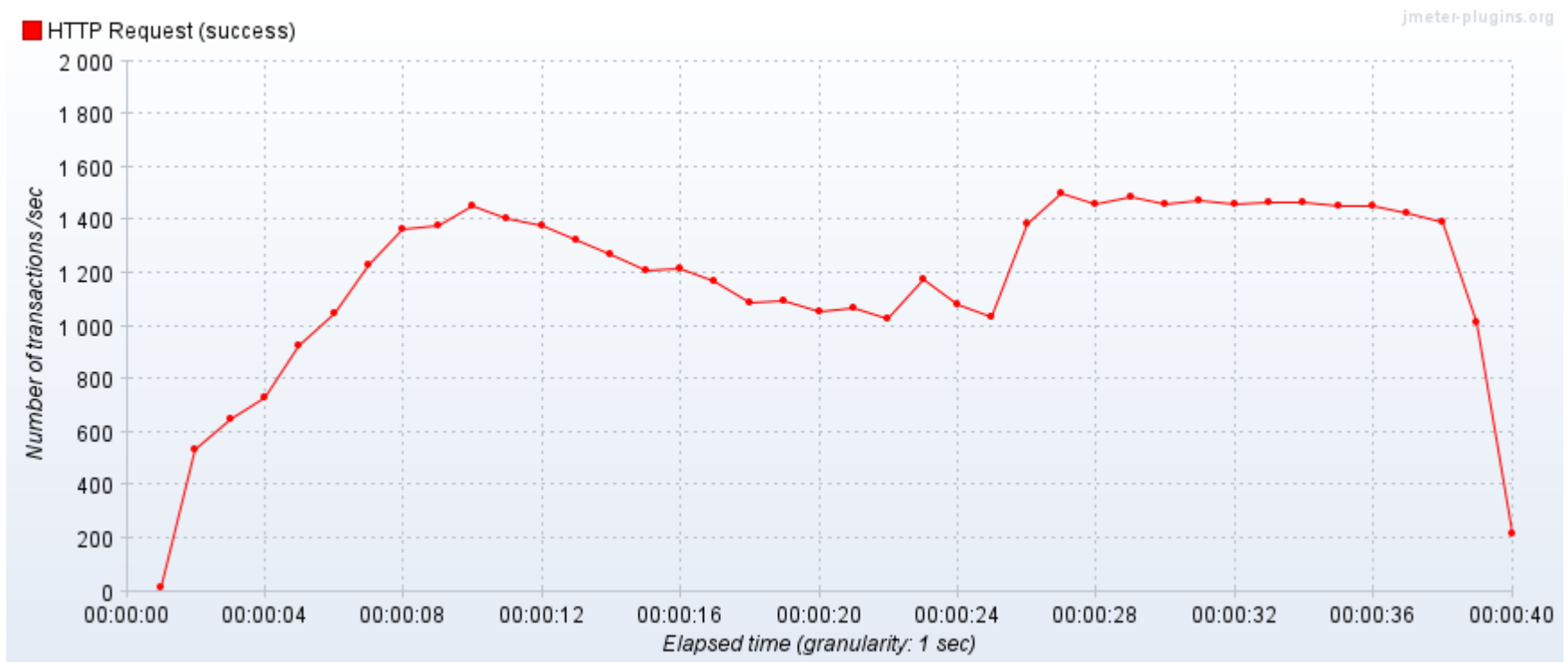
Add from Clipboard

Delete

Up

Down

Resultado



<https://medium.com/@daniduwijendra/resolving-java-net-bindexception-e119f284518f>